

Laporan Tugas Besar

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 4: INTERMEDIATE CODE & INTERPRETER

SEMESTER II / 2025–2026



Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021

Nicholas Wise Saragih Sumbayak - 13524037

Kurt Mikhael Purba - 13524065

Rava Khoman Tuah Saragih - 13524107

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

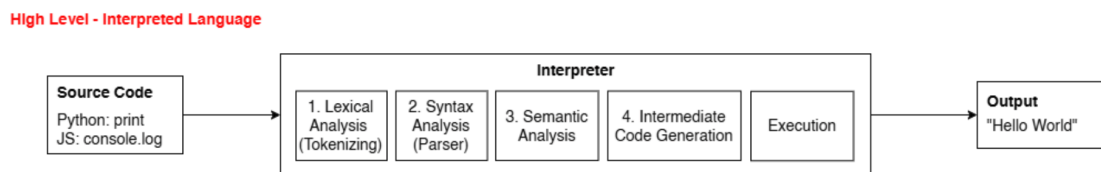
1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	4
1.3	Gambaran Umum Sistem	4
1.4	Spesifikasi Umum	5
2	Dasar Teori	5
2.1	Pendefinisian Bahasa Pemrograman	5
2.1.1	Hierarki Bahasa Pemrograman	5
2.1.2	<i>Compiler</i> dan <i>Interpreter</i>	6
2.2	Teori Kompilator (<i>Compiler Theory</i>)	6
2.2.1	<i>Intermediate Code Generation</i>	7
2.2.2	<i>Stack Machine</i>	8
2.2.3	<i>Display</i>	9
2.3	Teori Bahasa Formal dan Otomata	9
2.3.1	Kaitannya dengan Milestone 1–3	9
2.3.2	<i>Three-Address Code</i> dan Mesin Abstrak	10
3	Perancangan	10
3.1	Perancangan <i>Intermediate Code Generator</i>	10
3.1.1	Struktur Data Instruksi	11
3.1.2	Algoritma <i>Code Generation</i>	11
3.2	Perancangan <i>Stack Machine</i>	12
3.3	Perancangan <i>Interpreter</i>	12
3.4	Perancangan Penanganan <i>Runtime Error</i>	13
4	Implementasi	13
4.1	Struktur Direktori Proyek	13
4.2	Implementasi <code>Instruction.hpp</code> dan <code>Instruction.cpp</code>	14
4.3	Implementasi <code>CodeGenerator</code>	15
4.3.1	Expression dan Assignment	15
4.3.2	Kontrol Alur (If, While, For, Repeat)	15
4.3.3	Subprogram (Procedure dan Function)	16
4.4	Implementasi <code>StackMachine</code>	16
4.5	Implementasi <code>Interpreter</code>	17
5	Pengujian	18
5.1	Pengujian Unit	18
5.2	Pengujian Integrasi	18
5.2.1	Kasus Uji 1: Basic	18
5.2.2	Kasus Uji 2: Aritmetika	19
5.2.3	Kasus Uji 3: If-Else	20
5.2.4	Kasus Uji 4: While	21
5.2.5	Kasus Uji 5: For Loop	22
5.2.6	Kasus Uji 6: Nested (While + If)	22
5.2.7	Kasus Uji 7: Repeat-Until	24
5.2.8	Kasus Uji 8: Prosedur dengan Parameter	25

5.2.9 Kasus Uji 9: Negatif – Division by Zero	26
5.2.10 Kasus Uji 10: Negatif – Integer Overflow	26
5.2.11 Kasus Uji 11: Array dan Record	26
5.2.12 Kasus Uji 12: Case Statement	28
5.2.13 Kasus Uji 13: Function dengan Return Value	29
5.2.14 Kasus Uji 14: Boolean Literal	30
6 Fitur Tambahan	30
7 Penutup	31
7.1 Kesimpulan	31
7.2 Saran	31
Appendix	32
Pembagian Tugas	32

1 Pendahuluan

Pada tahap sebelumnya, telah berhasil dibangun sebuah *semantic analyzer* yang mampu mengkonversi *Parse Tree* menjadi *Decorated Abstract Syntax Tree* (Decorated AST) serta melakukan pengecekan semantik seperti kecocokan tipe data, *scope resolution*, dan deteksi *identifier* yang belum dideklarasikan. Meskipun *semantic analyzer* telah berfungsi dengan baik dalam memvalidasi makna program, tahapan kompilasi belum mencapai titik di mana program dapat dijalankan. Hasil dari tahap analisis semantik berupa *Decorated AST* beserta *symbol table* masih berupa representasi abstrak yang belum dapat dieksekusi secara langsung oleh mesin.

Proses transformasi kode sumber menjadi program yang dapat dijalankan melibatkan beberapa tahap berurutan. Setelah analisis semantik selesai, tahap berikutnya adalah pembangkitan kode perantara (*intermediate code generation*) dan pengeksekusian kode tersebut (*execution*). Pada tahap pembangkitan kode, *intermediate code generator* menerima *Decorated AST* dari *semantic analyzer* dan menghasilkan representasi kode perantara berupa *Three-Address Code* (TAC). Kode perantara ini menjadi jembatan antara representasi bahasa tingkat tinggi dan mesin eksekusi. Pada tahap eksekusi, *interpreter* berbasis *stack machine* mengeksekusi instruksi-instruksi TAC untuk menghasilkan keluaran program yang diharapkan.



Gambar 1: Pipeline kompilasi lengkap hingga Milestone 4

Sumber: Spesifikasi Milestone 4 Tugas Besar IF2224

1.1 Latar Belakang

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik (*syntax analysis*), analisis semantik (*semantic analysis*), pembangkitan kode perantara (*intermediate code generation*), hingga eksekusi. Setelah *semantic analyzer* berhasil memvalidasi makna program dan menghasilkan *Decorated AST*, tahap selanjutnya adalah mengubah representasi tersebut menjadi instruksi-instruksi yang dapat dieksekusi oleh mesin.

Tahap pembangkitan kode perantara merupakan fase keempat dalam rantai kompilasi. Fase ini bertugas untuk mentransformasi *Decorated AST* menjadi bentuk perantara yang lebih dekat ke bahasa mesin namun masih bersifat abstrak dan independen terhadap arsitektur mesin target. Salah satu bentuk representasi perantara yang umum digunakan adalah *Three-Address Code* (TAC), yang setiap instruksinya umumnya memiliki bentuk $x = y \text{ op } z$ yang melibatkan paling banyak tiga alamat: dua operand sumber dan satu alamat tujuan.

Setelah kode perantara dihasilkan, diperlukan sebuah mesin eksekusi untuk menjalankan instruksi-instruksi tersebut. Dalam konteks tugas besar ini, mesin eksekusi yang dibangun adalah *stack machine* yang menggunakan struktur data *stack* sebagai media penyimpanan utama. Setiap subprogram yang dipanggil akan memiliki *frame* sendiri di

dalam stack, yang berisi *static link*, *dynamic link*, *return address*, serta ruang untuk variabel lokal dan parameter. Mekanisme *display* digunakan untuk mengelola *static scope* dan mempercepat akses variabel dari *scope* induk.

Pada tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata, mahasiswa diminta untuk membangun sebuah *compiler* sederhana untuk bahasa pemrograman Arion, yaitu bahasa bertipe Pascal. Milestone keempat dan terakhir ini berfokus pada pembangunan *intermediate code generator* yang mengonversi *Decorated AST* menjadi instruksi *Three-Address Code* (TAC), serta *interpreter* berbasis *stack machine* yang mengeksekusi instruksi-instruksi tersebut untuk menjalankan program.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan:

1. **Intermediate Code Generator** yang mampu mengonversi *Decorated AST* menjadi instruksi *Three-Address Code* (TAC) yang mencakup ekspresi, penugasan, kontrol alur (*if*, *while*, *for*, *repeat*), subprogram (*procedure/function*), dan I/O.
2. **Stack Machine** yang mampu mengelola memori stack, *frame* untuk setiap pemanggilan subprogram, *display* untuk resolusi *scope*, serta mendeteksi dan menangani kesalahan *runtime* seperti *stack overflow*, pembagian dengan nol, dan *jump* tidak valid.
3. **Interpreter** yang mampu mengeksekusi daftar instruksi TAC dalam siklus *Fetch-Decode-Execute* serta menjalankan seluruh operasi aritmetika, logika, relasional, dan I/O.

1.3 Gambaran Umum Sistem

Sistem yang dibangun pada Milestone 4 merupakan bagian akhir dari rantai kompilasi bahasa Arion yang diimplementasikan dalam bahasa C++20. Sistem ini menerima *Decorated AST* dan *symbol table* dari tahap analisis semantik, menghasilkan instruksi TAC, dan mengeksekusinya menggunakan mesin *stack*. Sistem terdiri dari dua komponen utama:

1. **Intermediate Code Generator** (`src/intermediate/`): Menerima *Decorated AST* dan *symbol table* dari tahap analisis semantik, melakukan traversal pada AST untuk menghasilkan daftar instruksi TAC. *Code generator* dirancang menggunakan pola *Visitor* dengan fungsi `visit()` untuk setiap jenis node AST.
2. **Runtime Environment** (`src/runtime/`): Terdiri dari **StackMachine** (manajemen stack dan frame) dan **Interpreter** (siklus eksekusi instruksi). Interpreter membaca instruksi TAC satu per satu, mendekode opcode, dan mengeksekusi operasi yang sesuai pada stack menggunakan siklus *Fetch-Decode-Execute*.

Keluaran akhir sistem terdiri dari:

- **Intermediate Code**: Daftar instruksi TAC yang dihasilkan dari proses *code generation*, berupa baris-baris dengan format `<line> <INSTR> <level> <operand>`.
- **Program Output**: Hasil eksekusi program yang dicetak ke `stdout` melalui instruksi `WRT` dan `WRTLN`, termasuk keluaran dari operasi I/O yang didefinisikan dalam program sumber.

1.4 Spesifikasi Umum

1. Intermediate Code Generation:

- Mengubah *Decorated AST* menjadi instruksi TAC dengan format `<line> <INSTR> <level> <operand>`.
- Mendukung 9 opcode: INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET.
- Mendukung ekspresi, penugasan, I/O (`write`, `writeln`, `read`, `readln`).
- Mendukung kontrol alur: `if-then-else`, `while-do`, `for-to/downto`, `repeat-until`.
- Mendukung subprogram: prosedur dan fungsi dengan parameter.
- Manajemen *label* untuk *backpatching* instruksi lompatan.

2. Stack Machine & Interpreter:

- Manajemen stack dengan struktur *frame* per pemanggilan subprogram (SL, DL, RA, variabel lokal).
- Mekanisme *display* untuk resolusi *lexical level* (*static scope*).
- Siklus eksekusi *Fetch-Decode-Execute* untuk menjalankan instruksi TAC.
- 14 operasi OPR: NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ, WRT, WRTLN.
- Penanganan kesalahan *runtime*: *stack overflow/underflow*, pembagian nol, *jump* tidak valid, akses di luar batas, dan korupsi stack.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks (*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika atau ketidakcocokan tipe data. Pada tahap ini, setelah program dinyatakan benar secara sintaksis dan semantik, program perlu ditransformasi menjadi instruksi-instruksi yang dapat dieksekusi oleh mesin.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- **Low-Level Language (Bahasa Tingkat Rendah):** Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.

- **High-Level Language (Bahasa Tingkat Tinggi):** Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler dan Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. **Compiler (Kompilator):** *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. **Interpreter:** Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*. Pada Milestone 4, *interpreter* yang dibangun bekerja pada level instruksi TAC, bukan pada kode sumber asli.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. **Lexical Analysis:** Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber. Fase ini membaca karakter demi karakter dan mengelompokkannya menjadi token-token bermakna. Penjelasan lebih detail mengenai fase ini dibahas pada laporan Milestone 1.
2. **Syntax Analysis (Parsing):** Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*). Fase ini memeriksa kebenaran susunan *token* berdasarkan aturan tata bahasa (*grammar*) kode sumber. Penjelasan lebih detail mengenai fase ini dibahas pada laporan Milestone 2.
3. **Semantic Analysis:** Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan

identifier telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi (*Decorated AST*). Penjelasan lebih detail mengenai fase ini dibahas pada laporan Milestone 3.

4. **Intermediate Code Generation:** Menghasilkan representasi kode perantara untuk mesin abstrak, yang posisinya menjembatani antara bahasa tingkat tinggi dan bahasa mesin.
5. **Code Optimization:** Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi. Tujuannya adalah untuk mempercepat eksekusi program tanpa membuang sumber daya (*CPU*, memori).
6. **Target Code Generation:** Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Intermediate Code Generation

Sebagai fokus utama pada *milestone* ini, *Intermediate Code Generation* adalah fase keempat dari kompilator yang mentransformasi representasi semantik program (*Decorated AST*) menjadi bentuk perantara yang bersifat abstrak dan independen terhadap mesin target. Fase ini memerlukan *Decorated AST* yang telah dianotasi dengan informasi tipe data, *scope*, dan alamat memori dari *symbol table* agar dapat menghasilkan instruksi yang sesuai.

Salah satu bentuk representasi antara yang umum digunakan adalah *Three-Address Code* (TAC). TAC disebut *three-address code* karena setiap instruksi umumnya memiliki bentuk $x = y \text{ op } z$, yang melibatkan tiga alamat: dua operand sumber dan satu alamat tujuan. Namun, dalam implementasi proyek ini, TAC direpresentasikan dalam format yang lebih sederhana dan lebih dekat ke arsitektur mesin stack:

```
<line> <OPCODE> <level> <operand>
```

Setiap instruksi terdiri dari empat komponen:

- **line:** Nomor baris instruksi (berbasis 0, diisi secara otomatis saat pembuatan instruksi).
- **OPCODE:** Kode operasi yang menentukan jenis instruksi (INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET).
- **level:** Tingkat *lexical level* yang digunakan untuk resolusi *scope* variabel melalui mekanisme *display*.
- **operand:** Nilai literal, alamat memori, nomor operasi, atau baris target lompatan, tergantung pada jenis instruksi.

Daftar Instruksi TAC:

Tabel 1: Daftar instruksi TAC

Opcode	Operasi	Semantik
INT m	Initiate Memory	Alokasi frame sebesar m word di stack
LIT v	Load Literal	Push nilai literal v ke stack
LOD 1 a	Load Value	Push value dari address a pada level 1
STO 1 a	Store Value	Pop stack lalu simpan ke address a pada level 1
CAL 1 a	Call	Panggil subprogram pada instruction line a
JMP 1 a	Unconditional Jump	IP = a
JPC 1 a	Conditional Jump	Jika $\text{pop}() == 0$, IP = a
OPR 1 a	Operation	Eksekusi operasi a pada stack
RET 1 a	Return	Kembali dari subprogram

Operasi OPR (1–16):

Tabel 2: Daftar operasi OPR

No	Nama	Stack Effect
1	NEG	Pop $a \rightarrow$ Push $(-a)$
2	ADD	Pop $b, a \rightarrow$ Push $(a+b)$
3	SUB	Pop $b, a \rightarrow$ Push $(a-b)$
4	MUL	Pop $b, a \rightarrow$ Push $(a*b)$
5	DIV	Pop $b, a \rightarrow$ Push (a/b) – cek div by zero
6	MOD	Pop $b, a \rightarrow$ Push $(a \bmod b)$
7	EQL	Push $(a == b ? 1 : 0)$
8	NEQ	Push $(a != b ? 1 : 0)$
9	LSS	Push $(a < b ? 1 : 0)$
10	GEQ	Push $(a >= b ? 1 : 0)$
11	GTR	Push $(a > b ? 1 : 0)$
12	LEQ	Push $(a <= b ? 1 : 0)$
13	WRT	Pop $\rightarrow$ print tanpa newline
14	WRTLN	Pop $\rightarrow$ print dengan newline
15	ODD	Pop $a \rightarrow$ Push $(a \% 2 == 1 ? 1 : 0)$
16	NOT	Pop $a \rightarrow$ Push $(a == 0 ? 1 : 0)$

2.2.2 Stack Machine

Stack Machine adalah model mesin abstrak yang menggunakan struktur data *stack* sebagai media penyimpanan utama untuk operasi komputasi. Model ini umum digunakan dalam implementasi mesin virtual dan bahasa bytecode (misalnya JVM dan CLR) karena kesederhanaannya dalam manajemen operand dan pemanggilan fungsi. Setiap subprogram yang dipanggil akan memiliki *frame* sendiri di dalam stack, yang berfungsi sebagai ruang memori lokal untuk variabel, parameter, dan informasi kontrol pemanggilan.

Layout sebuah *frame* pada stack adalah sebagai berikut:

Tabel 3: Layout frame stack

Address Relatif	Isi	Keterangan
$-k$ s.d. -1	Parameter ke-1 s.d. ke- k	Argumen yang didorong sebelum CAL
0	Static Link (SL)	Base pointer frame induk (<i>lexical scope</i>)
1	Dynamic Link (DL)	Base pointer frame pemanggil
2	Return Address (RA)	Indeks instruksi tujuan setelah RET
3	Variabel lokal ke-1	
4	Variabel lokal ke-2	
...	...	
$3 + (n - 1)$	Variabel lokal ke- n	

Penting untuk dicatat bahwa parameter tidak dialokasikan di dalam frame baru yang dibuat oleh **INT**. Sebaliknya, parameter berada di stack tepat di bawah base pointer frame baru, sehingga diakses dengan alamat negatif relatif terhadap **bp**. Hal ini memungkinkan argumen yang didorong oleh pemanggil sebelum instruksi **CAL** dapat diakses langsung oleh subprogram yang dipanggil. Untuk level 0 (program utama), *static link* diatur ke 0. Untuk fungsi atau prosedur di level L , *static link* diisi dengan `display[L - 1]`, yaitu base pointer dari frame terakhir pada level $L - 1$.

Ketika sebuah subprogram dipanggil, instruksi **CAL** menyebabkan penyimpanan return address dan pembuatan frame baru. Instruksi **INT** kemudian mengalokasikan ruang untuk variabel lokal dengan menambah *stack pointer*. Ketika subprogram selesai, instruksi **RET** memulihkan *base pointer*, *instruction pointer*, dan *display* ke kondisi sebelum pemanggilan, lalu melanjutkan eksekusi di instruksi setelah **CAL**.

2.2.3 Display

Display adalah mekanisme untuk mengelola *static scope* dalam bahasa pemrograman dengan *lexical scoping* seperti Pascal. Pada bahasa dengan *lexical scoping*, ruang lingkup (*scope*) sebuah variabel ditentukan oleh posisi deklarasinya dalam struktur bersarang program, bukan oleh urutan pemanggilan pada saat runtime. *Display* merupakan array yang menyimpan *base pointer* dari *frame* terakhir untuk setiap *lexical level*. Ketika interpreter perlu mengakses variabel dari *scope* induk, ia menggunakan nilai `display[level]` untuk mendapatkan base pointer dari frame pada level yang diinginkan, tanpa harus menelusuri *static link chain* secara iteratif.

Implementasi *display* dalam proyek ini terdiri dari sebuah array berukuran tetap (sesuai jumlah level maksimum) yang diperbarui setiap kali instruksi **INT** atau **RET** dieksekusi. Saat **INT** membuat frame baru pada level L , `display[L]` diset ke base pointer frame baru tersebut. Saat **RET** mengembalikan kontrol dari level L , `display[L]` dikembalikan ke nilai semula menggunakan informasi yang telah disimpan sebelumnya. Mekanisme ini memastikan bahwa variabel non-lokal dapat diakses dengan kompleksitas waktu $O(1)$, tidak tergantung pada kedalaman penelusuran *static link chain*.

2.3 Teori Bahasa Formal dan Otomata

2.3.1 Kaitannya dengan Milestone 1–3

Milestone 4 tidak memperkenalkan konsep baru dalam Teori Bahasa Formal dan Otomata secara langsung, melainkan memanfaatkan hasil dari milestone-milestone sebelum-

nya sebagai fondasi untuk pembangkitan dan eksekusi kode. Konsep-konsep yang telah dibangun pada milestone sebelumnya adalah sebagai berikut:

- **Milestone 1:** *Regular Expression* dan Deterministic Finite Automaton (DFA) digunakan untuk mengenali pola-pola leksikal dalam kode sumber, menghasilkan token-token yang menjadi masukan parser.
- **Milestone 2:** *Context-Free Grammar* (CFG) dan algoritma *Recursive Descent* digunakan untuk memeriksa struktur sintaks dan membangun *Parse Tree*.
- **Milestone 3:** *Attributed Grammar* dan *Visitor Pattern* digunakan untuk melakukan analisis semantik, mengelola *symbol table*, dan menghasilkan *Decorated AST* yang menjadi masukan untuk *intermediate code generator* pada milestone ini.

Decorated AST yang dihasilkan dari Milestone 3 merupakan representasi program yang telah dianotasi dengan informasi semantik lengkap, termasuk tipe data setiap node, referensi ke entri *symbol table*, dan tingkat *lexical level*. Informasi ini sangat penting bagi *code generator* untuk menentukan alamat memori, level leksikal, dan jenis operasi yang tepat untuk setiap konstruksi program.

2.3.2 *Three-Address Code* dan Mesin Abstrak

Dalam konteks teori bahasa formal, TAC dapat dipandang sebagai bahasa dari sebuah mesin abstrak (*abstract machine*) yang memiliki stack sebagai memori utamanya. Bahasa ini didefinisikan oleh sekumpulan instruksi (opcode) dan aturan transisi status mesin (semantik operasional). Setiap instruksi TAC merupakan sebuah simbol terminal dalam bahasa mesin abstrak ini, dan urutan instruksi membentuk sebuah *string* yang diterima oleh mesin.

Mesin abstrak stack dapat dimodelkan secara formal sebagai sistem transisi status dengan state berupa tupel (S, ip, D, bp) , di mana S adalah stack, ip adalah *instruction pointer*, D adalah *display*, dan bp adalah *base pointer*. Setiap instruksi menyebabkan transisi dari satu state ke state berikutnya sesuai dengan semantik operasionalnya, membentuk jejak eksekusi (*execution trace*) program. Sebagai contoh, instruksi LIT v menyebabkan transisi $(S, ip, D, bp) \rightarrow (S \cup \{v\}, ip + 1, D, bp)$, yaitu nilai v ditambahkan ke stack dan *instruction pointer* maju satu langkah.

Pemodelan formal ini memastikan bahwa eksekusi program bersifat deterministik: untuk setiap state dan instruksi yang sedang dieksekusi, terdapat tepat satu state berikutnya yang valid. Sifat deterministik ini merupakan konsekuensi dari arsitektur *von Neumann* yang mendasari desain mesin stack, di mana setiap instruksi memiliki efek yang terdefinisi dengan jelas terhadap status mesin.

3 Perancangan

3.1 Perancangan *Intermediate Code Generator*

Code generator dirancang dengan pola desain *Visitor* (*ASTVisitor*). Setiap jenis node AST memiliki fungsi `visit()` yang menghasilkan instruksi TAC yang sesuai. *Code generator* berjalan dalam satu lintasan (*single pass*) pada AST yang sudah dianotasi.

3.1.1 Struktur Data Instruksi

Setiap instruksi TAC direpresentasikan oleh struktur berikut:

Listing 1: Struktur Instruction

```

1 struct Instruction {
2     int line;           // instruction index (0-based, auto-set)
3     Opcode opcode;     // INT, LIT, LOD, STO, CAL, JMP, JPC, OPR
                        // , RET
4     int level;         // static nesting level
5     int operand;       // literal value, address, operation
                        // number, etc.
6 };

```

3.1.2 Algoritma Code Generation

Berikut adalah algoritma *code generation* untuk setiap node AST:

- **ProgramNode**: Hitung `frameSize` dari `btabs`, emit `INT`, proses deklarasi, proses body, emit `RET`.
- **AssignNode**: `visit(value)`, emit `STO` dengan `level` dan `adr` dari target.
- **BinaryOpNode**: `visit(left)`, `visit(right)`, emit `OPR` dengan nomor operasi sesuai operator.
- **UnaryOpNode**: `visit(operand)`, emit `OPR` `NEG` atau `NOT`.
- **LiteralNode**: Emit `LIT` dengan nilai literal.
- **VariableNode**: Emit `LOD` dengan `level` dan `adr` dari symbol table.
- **IfNode**: Evaluasi kondisi, `JPC` ke `else-label`, `then-branch`, `JMP` ke `end-label`, `else-branch`.
- **WhileNode**: Place `start-label`, kondisi, `JPC` ke `end-label`, body, `JMP` ke `start-label`.
- **RepeatNode**: Place `start-label`, statements, kondisi, `JPC` ke `start-label`.
- **ForNode**: Init, `STO`, `start-label`, check condition (`LEQ/GEQ`), `JPC`, body, increment/decrement, `JMP`.
- **ProcDeclNode**: Catat `adr`, `INT`, lokal, body, `RET`.
- **FuncDeclNode**: Sama dengan `ProcDeclNode`.
- **ProcCallNode**: Untuk I/O: push argumen, `OPR` `WRT/WRTLN`. Untuk user-defined: push argumen, `CAL`.

3.2 Perancangan *Stack Machine*

Stack machine dirancang dengan operasi-operasi berikut:

- `push(value)`: Push nilai ke stack.
- `pop()`: Pop nilai dari stack (dengan deteksi underflow).
- `pushFrame(frameSize, staticLink, returnAddr)`: Alokasi frame baru dengan SL, DL, RA, dan inisialisasi variabel lokal.
- `popFrame()`: Pop seluruh frame dan kembalikan BP.
- `store(level, addr, value)`: Simpan value pada address di frame level tertentu melalui *static link chain*.
- `load(level, addr)`: Load value dari address di frame level tertentu.
- `setDisplay/getDisplay`: Manajemen display untuk scope.

3.3 Perancangan *Interpreter*

Interpreter dirancang dengan siklus *Fetch-Decode-Execute*:

Listing 2: Siklus utama interpreter

```

1 void Interpreter::execute(const vector<Instruction> &
  instructions) {
2     ip = 0;
3     while (ip >= 0 && ip < instructions.size()) {
4         const Instruction &inst = instructions[ip];
5         try {
6             switch (inst.opcode) {
7                 case INT: handleINT(inst); break;
8                 case LIT: handleLIT(inst); break;
9                 // ... semua opcode
10            }
11        } catch (const RuntimeError &e) {
12            errors.push_back(e);
13            return;
14        }
15        ip++;
16    }
17 }
```

Setiap handler instruksi didefinisikan secara terpisah:

- **handleINT**: Push Frame baru dengan static link dari display.
- **handleLIT**: Push `inst.operand` ke stack.
- **handleLOD**: Load value dari address dan push ke stack.
- **handleSTO**: Pop stack dan store ke address.

- **handleJMP**: Set `ip = operand - 1`.
- **handleJPC**: Pop condition; jika 0, set `ip = operand - 1`.
- **handleCAL**: Simpan return address dan set `ip = operand - 1`.
- **handleRET**: Pop frame dan set `ip = returnAddress - 1`.
- **handleOPR**: Dispatch ke 14 operasi OPR sesuai nomor operand.

3.4 Perancangan Penanganan *Runtime Error*

Sistem dirancang untuk mendeteksi dan melaporkan *runtime error* tanpa menyebabkan *crash*:

- **RuntimeError**: Struktur yang menyimpan tipe error, pesan, dan nomor baris instruksi tempat error terjadi.
- **Stack Overflow**: Deteksi saat `pushFrame()` melebihi `maxFrames` (default 1000).
- **Stack Underflow**: Deteksi saat `pop()` pada stack kosong.
- **Division By Zero**: Deteksi pada OPR DIV dan MOD.
- **Invalid Jump**: Validasi target JMP dan JPC berada dalam rentang instruksi.
- **Index Out of Bounds**: Validasi address absolut pada `store()` dan `load()`.

4 Implementasi

Intermediate code generator dan *interpreter* pada compiler Arion diimplementasikan dalam bahasa C++20. Kedua komponen ini berada pada direktori `src/intermediate/` dan `src/runtime/`.

4.1 Struktur Direktori Proyek

Listing 3: Tata letak direktori proyek

```

1 HBB-Tubes-IF2224-2026/
2   src/
3       main.cpp                # Titik masuk program
4       driver_m4.cpp           # Driver khusus Milestone
5           4
6       lexical/                # Lexical analyzer (M1)
7       syntax/                 # Syntax analyzer (M2)
8       semantic/               # Semantic analyzer (M3)
9       intermediate/           # Code generator (M4 --
10          BARU)
11           Instruction.hpp/cpp  # Opcode enum +
12                                   Instruction struct
13           CodeGenerator.hpp/cpp # Kelas CodeGenerator (
14                                   ASTVisitor)

```

```

11         runtime/                                # Runtime environment (M4
           -- BARU)
12         RuntimeError.hpp                          # Struktur RuntimeError
13         StackMachine.hpp/cpp                     # Kelas StackMachine
14         Interpreter.hpp/cpp                      # Kelas Interpreter
15     test/
16         milestone4/
17             input/                                # Berkas uji masukan M4
18                 pos_basic.txt
19                 pos_arith.txt
20                 pos_if_else.txt
21                 pos_while.txt
22                 pos_for.txt
23                 ...
24             output/                              # Keluaran yang
                diharapkan
25                 pos_basic.txt
26                 ...
27     doc/
28         m4/                                        # Laporan M4
29             main.tex
30             *.png                                # Screenshot hasil uji
31     Makefile                                     # Skrip kompilasi (
                diperbarui)

```

4.2 Implementasi Instruction.hpp dan Instruction.cpp

Berkas Instruction.hpp mendefinisikan:

- enum class Opcode: INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET.
- struct Instruction: line, opcode, level, operand.
- std::string toString() const: format "<line> <OPCODE> <level> <operand>".

Listing 4: Deklarasi Instruction

```

1  enum class Opcode {
2      INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET
3  };
4
5  struct Instruction {
6      int line;
7      Opcode opcode;
8      int level;
9      int operand;
10
11      std::string toString() const;
12  };

```

4.3 Implementasi CodeGenerator

Kelas CodeGenerator mengimplementasikan ASTVisitor dan menyediakan fungsi generate() sebagai titik masuk.

4.3.1 Expression dan Assignment

Listing 5: Implementasi ekspresi dan assignment

```

1 void CodeGenerator::visit(LiteralNode &node) {
2     int value = 0;
3     try { value = stoi(node.value); }
4     catch (...) {
5         if (node.value == "true") value = 1;
6         else if (node.value == "false") value = 0;
7     }
8     emit(LIT, 0, value);
9 }
10
11 void CodeGenerator::visit(VariableNode &node) {
12     const TabEntry &entry = symTable.getTab(node.tabIndex);
13     emit(LOD, entry.lev, entry.adr);
14 }
15
16 void CodeGenerator::visit(BinaryOpNode &node) {
17     if (node.left) node.left->accept(*this);
18     if (node.right) node.right->accept(*this);
19     int opNum = opToCode(node.op);
20     if (opNum > 0) emit(OPR, 0, opNum);
21 }
22
23 void CodeGenerator::visit(AssignNode &node) {
24     if (node.value) node.value->accept(*this);
25     if (node.target && node.target->tabIndex >= 0) {
26         const TabEntry &entry = symTable.getTab(node.target->
27             tabIndex);
28         emit(STO, entry.lev, entry.adr);
29     }
30 }

```

4.3.2 Kontrol Alur (If, While, For, Repeat)

Listing 6: Implementasi IfNode dengan backpatching

```

1 void CodeGenerator::visit(IfNode &node) {
2     int elseLabel = newLabel();
3     int endLabel = newLabel();
4
5     if (node.condition) node.condition->accept(*this);
6     int jpcIdx = instructions.size();
7     emit(JPC, 0, 0); // backpatched

```



```

8     if (node.thenBranch) node.thenBranch->accept(*this);
9
10    int jmpIdx = -1;
11    if (node.elseBranch) {
12        jmpIdx = instructions.size();
13        emit(JMP, 0, 0); // backpatched
14    }
15
16    placeLabel(elseLabel);
17    backpatch(jpcIdx, getLabelLine(elseLabel));
18    if (node.elseBranch) node.elseBranch->accept(*this);
19
20    placeLabel(endLabel);
21    if (jmpIdx >= 0) backpatch(jmpIdx, getLabelLine(endLabel)
22        );
23 }

```

4.3.3 Subprogram (Procedure dan Function)

Listing 7: Implementasi deklarasi prosedur

```

1 void CodeGenerator::visit(ProcDeclNode &node) {
2     TabEntry &entry = symTable.getTab(symTable.lookup(node.
3         name));
4     entry.adr = instructions.size(); // catat entry point
5
6     int blockIdx = symTable.getDisplay(entry.lev);
7     BTabEntry &block = symTable.getBTab(blockIdx);
8     emit(INT, entry.lev, 3 + block.psize + block.vsize);
9
10    for (auto &decl : node.localDeclarations) {
11        if (decl) decl->accept(*this);
12    }
13    if (node.body) node.body->accept(*this);
14    emit(RET, 0, 0);
15 }

```

4.4 Implementasi StackMachine

StackMachine mengelola memori stack dan frame:

Listing 8: Implementasi pushFrame dan popFrame

```

1 void StackMachine::pushFrame(int frameSize, int staticLink,
2     int returnAddr) {
3     if (currentFrames >= maxFrames) {
4         throw RuntimeError(STACK_OVERFLOW, "Stack overflow...");
5     }
6     push(staticLink); // SL
7     push(bp); // DL

```

```

7     push(returnAddr); // RA
8     for (int i = 3; i < frameSize; i++) push(0); // lokal
9     bp = sp - frameSize + 1;
10    currentFrames++;
11 }
12
13 void StackMachine::popFrame() {
14     if (currentFrames <= 0) {
15         throw RuntimeError(STACK_CORRUPTION, "No frame to pop
16                             ");
17     }
18     int dynLink = stack[bp + 1];
19     int frameSize = sp - bp + 1;
20     for (int i = 0; i < frameSize; i++) pop();
21     bp = dynLink;
22     currentFrames--;
23 }

```

4.5 Implementasi Interpreter

Interpreter memiliki siklus utama `execute()` dan handler untuk setiap jenis instruksi:

Listing 9: Implementasi OPR operations

```

1 void Interpreter::handleOPR(const Instruction &inst) {
2     switch (inst.operand) {
3         case 1: { // NEG
4             int a = stack.pop(); stack.push(-a); break;
5         }
6         case 2: { // ADD
7             int b = stack.pop(), a = stack.pop();
8             stack.push(a + b); break;
9         }
10        case 3: { // SUB
11            int b = stack.pop(), a = stack.pop();
12            stack.push(a - b); break;
13        }
14        // ... case 4--14 ...
15        case 13: { // WRT
16            int a = stack.pop(); cout << a; break;
17        }
18        case 14: { // WRTLN
19            int a = stack.pop(); cout << a << endl; break;
20        }
21        default:
22            throw RuntimeError(UNKNOWN_INSTRUCTION, "...");
23    }
24 }

```

5 Pengujian

Pengujian dilakukan terhadap program dari dua sisi: pengujian unit (*unit test*) untuk komponen StackMachine dan Interpreter, serta pengujian integrasi (*integration test*) menggunakan driver M4 yang menjalankan pipeline kompilasi penuh.

5.1 Pengujian Unit

Pengujian unit untuk Role 3 (`test/test_role3.cpp`) mencakup:

- StackMachine push/pop dan deteksi underflow
- Frame push/pop dan akses memori
- Static link chain / frame bersarang
- Interpreter LIT + STO + LOD
- Interpreter JMP (unconditional jump)
- Interpreter JPC (conditional jump)
- Interpreter CAL + RET
- Invalid jump target
- Out-of-bounds memory access
- Stack overflow (max frames)
- Display management

5.2 Pengujian Integrasi

Bagian ini menyajikan hasil pengujian integrasi pada implementasi milestone 4 (M4). Terdapat 14 kasus uji yang mencakup berbagai fitur bahasa Pascal, mulai dari operasi dasar, kontrol alur, prosedur dan fungsi, hingga penanganan kesalahan runtime. Setiap kasus meliputi kode masukan, keluaran yang diharapkan, dan analisis terhadap kode perantara (intermediate code) yang dihasilkan.

5.2.1 Kasus Uji 1: Basic

Berkas: `pos_basic.txt`

Masukan:

```
1 program Basic;  
2 var x: integer;  
3 begin  
4     x := 42;  
5     writeln(x);  
6 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 42
3 STO 0 3
4 LOD 0 3
5 OPR 0 14
6 RET 0 0
```

=== Program Output ===

42

Execution completed successfully.

Analisis: Program dasar ini menguji jalur paling sederhana: deklarasi variabel, assignment literal, dan pemanggilan `writeln`. Instruksi `JMP 0 2` diletakkan setelah `INT` untuk melewati tubuh prosedur (jika ada) menuju badan utama program. Variabel `x` dialokasikan pada alamat 3 (frame header memakai 3 sel untuk SL, DL, RA).

5.2.2 Kasus Uji 2: Aritmetika

Berkas: `pos_arith.txt`

Masukan:

```
1 program Arith;
2 var x: integer;
3 begin
4     x := 10 + 20 * 3;
5     writeln(x);
6 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 10
3 LIT 0 20
4 LIT 0 3
5 OPR 0 4
6 OPR 0 2
7 STO 0 3
8 LOD 0 3
9 OPR 0 14
10 RET 0 0
```

=== Program Output ===

70

Execution completed successfully.

Analisis: Ekspresi $10 + 20 * 3$ dievaluasi dengan urutan postfix yang benar: $20 * 3 = 60$, lalu $10 + 60 = 70$. Prioritas operator (perkalian di atas penjumlahan) dipastikan oleh parser yang membangun AST yang tepat. Kode perantara memancarkan LIT 20, LIT 3, OPR 0 4 (MUL), lalu LIT 10, OPR 0 2 (ADD).

5.2.3 Kasus Uji 3: If-Else

Berkas: pos_if_else.txt

Masukan:

```

1 program IfElse;
2 var x, y: integer;
3 begin
4     x := 10;
5     if x > 5 then
6         y := 1
7     else
8         y := 0;
9     writeln(y);
10 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```

0 INT 0 5
1 JMP 0 2
2 LIT 0 10
3 STO 0 3
4 LOD 0 3
5 LIT 0 5
6 OPR 0 11
7 JPC 0 11
8 LIT 0 1
9 STO 0 4
10 JMP 0 13
11 LIT 0 0
12 STO 0 4
13 LOD 0 4
14 OPR 0 14
15 RET 0 0
```

=== Program Output ===

1

Execution completed successfully.

Analisis: Instruksi JPC 0 11 melompat ke cabang `else` jika kondisi $x > 5$ bernilai false. Karena $x = 10$, kondisi benar sehingga $y := 1$ dieksekusi, lalu JMP 0 13 melewati cabang `else`. Variabel x berada di alamat 3 dan y di alamat 4.

5.2.4 Kasus Uji 4: While

Berkas: pos_while.txt

Masukan:

```
1 program WhileLoop;
2 var i: integer;
3 begin
4     i := 5;
5     while i > 0 do
6     begin
7         writeln(i);
8         i := i - 1;
9     end;
10 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 5
3 STO 0 3
4 LOD 0 3
5 LIT 0 0
6 OPR 0 11
7 JPC 0 15
8 LOD 0 3
9 OPR 0 14
10 LOD 0 3
11 LIT 0 1
12 OPR 0 3
13 STO 0 3
14 JMP 0 4
15 RET 0 0
```

=== Program Output ===

```
5
4
3
2
1
```

Execution completed successfully.

Analisis: Loop `while` diimplementasikan dengan pola: (1) evaluasi kondisi di alamat 4, (2) `JPC 0 15` keluar jika kondisi salah, (3) badan loop pada alamat 8–13, (4) `JMP 0 4` kembali ke evaluasi kondisi. Variabel `i` (alamat 3) dikurangi 1 setiap iterasi menggunakan `OPR 0 3 (SUB)`.

5.2.5 Kasus Uji 5: For Loop

Berkas: pos_for.txt

Masukan:

```
1 program ForLoop;
2 var i: integer;
3 begin
4     for i := 1 to 5 do
5         begin
6             writeln(i);
7         end;
8 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 1
3 STO 0 3
4 LOD 0 3
5 LIT 0 5
6 OPR 0 12
7 JPC 0 15
8 LOD 0 3
9 OPR 0 14
10 LOD 0 3
11 LIT 0 1
12 OPR 0 2
13 STO 0 3
14 JMP 0 4
15 RET 0 0
```

=== Program Output ===

```
1
2
3
4
5
```

Execution completed successfully.

Analisis: Loop for i := 1 to 5 diterjemahkan menjadi pola while. Batas atas (5) di-LIT-kan dan dibandingkan dengan nilai i menggunakan OPR 0 12 (GTE). Jika sudah melebihi, JPC keluar dari loop. Inkrementasi dilakukan dengan OPR 0 2 (ADD) dan disimpan kembali ke alamat 3. Hasil menunjukkan iterasi berjalan dari 1 hingga 5 dengan benar.

5.2.6 Kasus Uji 6: Nested (While + If)

Berkas: pos_nested.txt

Masukan:

```
1 program NestedTest;
2 var
3   i: integer;
4 begin
5   i := 10;
6   while i > 0 do
7     begin
8       if i mod 2 == 0 then
9         writeln(i);
10      i := i - 1;
11    end;
12 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 10
3 STO 0 3
4 LOD 0 3
5 LIT 0 0
6 OPR 0 11
7 JPC 0 21
8 LOD 0 3
9 LIT 0 2
10 OPR 0 6
11 LIT 0 0
12 OPR 0 7
13 JPC 0 16
14 LOD 0 3
15 OPR 0 14
16 LOD 0 3
17 LIT 0 1
18 OPR 0 3
19 STO 0 3
20 JMP 0 4
21 RET 0 0
```

=== Program Output ===

```
10
8
6
4
2
```

Execution completed successfully.

Analisis: Kasus ini menguji kombinasi `while` dan `if` bersarang. Hanya nilai `i` yang

genap (10, 8, 6, 4, 2) yang dicetak. Ini memvalidasi bahwa JPC untuk if tidak mengganggu alur loop while. Penanganan mod menggunakan OPR 0 10 (MOD) dan perbandingan dengan OPR 0 7 (EQL).

5.2.7 Kasus Uji 7: Repeat-Until

Berkas: pos_repeat.txt

Masukan:

```
1 program RepeatTest;
2 var
3   i: integer;
4 begin
5   i := 10;
6   repeat
7     writeln(i);
8     i := i - 1;
9   until i == 0;
10 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4
1 JMP 0 2
2 LIT 0 10
3 STO 0 3
4 LOD 0 3
5 OPR 0 14
6 LOD 0 3
7 LIT 0 1
8 OPR 0 3
9 STO 0 3
10 LOD 0 3
11 LIT 0 0
12 OPR 0 7
13 JPC 0 4
14 RET 0 0
```

=== Program Output ===

```
10
9
8
7
6
5
4
3
2
1
```

Execution completed successfully.

Analisis: Loop `repeat-until` dieksekusi setidaknya sekali sebelum kondisi diperiksa. Kode perantara yang dihasilkan serupa dengan `while`, tetapi kondisi pemeriksaan berada di akhir badan loop. Variabel `i` diturunkan dari 10 hingga 1, dan semua nilai dicetak karena pemeriksaan `until i == 0` terjadi setelah penurunan.

5.2.8 Kasus Uji 8: Prosedur dengan Parameter

Berkas: `pos_procedure_param.txt`

Masukan:

```

1 program ProcParam;
2
3 var x: integer;
4
5 procedure printNum(n: integer);
6 begin
7     writeln(n);
8 end;
9
10 begin
11     x := 42;
12     printNum(x);
13 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```

0 INT 0 4
1 JMP 0 6
2 INT 1 4
3 LOD 1 -1
4 OPR 0 14
5 RET 0 0
6 LIT 0 42
7 STO 0 3
8 LOD 0 3
9 CAL 0 2
10 RET 0 0
```

=== Program Output ===

42

Execution completed successfully.

Analisis: Ini adalah kasus uji paling kritis karena menguji parameter passing. Parameter `n` diakses menggunakan alamat negatif `-1` (`LOD 1 -1`) karena argumen `x` sudah berada di stack sebelum `CAL`. Frame prosedur dialokasikan dengan `INT 1 4` (hanya untuk variabel lokal, tanpa parameter). Instruksi `JMP 0 6` di awal memastikan eksekusi tidak jatuh ke tubuh prosedur.

5.2.9 Kasus Uji 9: Negatif – Division by Zero

Berkas: neg_div_zero.txt

Masukan:

```
1 program DivZero;
2 var x: integer;
3 begin
4     x := 10;
5     x := x div 0;
6     writeln(x);
7 end.
```

Keluaran yang diharapkan:

Runtime error: Division by zero

Analisis: Interpreter mendeteksi pembagian dengan nol pada saat eksekusi OPR 0 5 (DIV) dan melempar `RuntimeError`. Error ini tertangkap oleh `driver_m4.cpp` dan ditampilkan ke pengguna. Deteksi runtime diperlukan karena nilai pembagi tidak dapat diprediksi pada saat kompilasi.

5.2.10 Kasus Uji 10: Negatif – Integer Overflow

Berkas: neg_overflow.txt

Masukan:

```
1 program Overflow;
2 var x: integer;
3 begin
4     x := 2147483647;
5     x := x + 1;
6     writeln(x);
7 end.
```

Keluaran yang diharapkan:

Runtime error: Integer overflow in addition

Analisis: Interpreter mendeteksi overflow aritmatika pada operasi OPR 0 2 (ADD). Nilai maksimum `integer` (32-bit signed) adalah 2147483647; penambahan 1 akan menyebabkan overflow. Deteksi ini mencegah perilaku undefined yang dapat merusak hasil komputasi selanjutnya.

5.2.11 Kasus Uji 11: Array dan Record

Berkas: pos_array_record.txt

Masukan:

```
1 program TestVar;
2
3 var
4     arr: array [1..10] of integer;
5     rec: record x: integer end;
```

```
6
7 begin
8   arr[1] := 5;
9   rec.x := 10;
10  arr[2] := arr[1] + rec.x;
11 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 14
1 JMP 0 2
2 LIT 0 1
3 LIT 0 1
4 LIT 0 10
5 OPR 0 17
6 LIT 0 1
7 OPR 0 3
8 LIT 0 5
9 STOA 0 3
10 LIT 0 0
11 LIT 0 10
12 STOA 0 13
13 LIT 0 2
14 LIT 0 1
15 LIT 0 10
16 OPR 0 17
17 LIT 0 1
18 OPR 0 3
19 LIT 0 1
20 LIT 0 1
21 LIT 0 10
22 OPR 0 17
23 LIT 0 1
24 OPR 0 3
25 LODA 0 3
26 LIT 0 0
27 LODA 0 13
28 OPR 0 2
29 STOA 0 3
30 RET 0 0
```

=== Program Output ===

Execution completed successfully.

Analisis: Menguji akses array (LODA, STOA) dan record field (STOA). Instruksi OPR 17 adalah BOUNDS_CHECK untuk memvalidasi indeks array. Array dialokasikan pada frame dengan ukuran 14 (10 elemen array + 4 field record).

5.2.12 Kasus Uji 12: Case Statement

Berkas: pos_case.txt

Masukan:

```
1 program TestCase;
2 var x, y: integer;
3 begin
4   x := 2;
5   case x of
6     1: y := 10;
7     2: y := 20;
8     3: y := 30;
9   end;
10  writeln(y);
11 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 5
1 JMP 0 2
2 LIT 0 2
3 STO 0 3
4 LOD 0 3
5 LIT 0 1
6 OPR 0 7
7 JPC 0 11
8 LIT 0 10
9 STO 0 4
10 JMP 0 25
11 LOD 0 3
12 LIT 0 2
13 OPR 0 7
14 JPC 0 18
15 LIT 0 20
16 STO 0 4
17 JMP 0 25
18 LOD 0 3
19 LIT 0 3
20 OPR 0 7
21 JPC 0 25
22 LIT 0 30
23 STO 0 4
24 JMP 0 25
25 LOD 0 4
26 OPR 0 14
27 RET 0 0
```

=== Program Output ===

20

Execution completed successfully.

Analisis: Menguji statement case dengan multiple branches. Setiap branch menggunakan JPC untuk membandingkan nilai x dengan konstanta case menggunakan EQL (OPR 7). Jika cocok, nilai y di-assign dan JMP melewati branch lainnya.

5.2.13 Kasus Uji 13: Function dengan Return Value

Berkas: test_func.txt

Masukan:

```

1 program FuncTest;
2 var x: integer;
3 function f(n: integer): integer;
4 begin
5     f := n * 2;
6 end;
7 begin
8     x := f(5);
9     writeln(x);
10 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```

0 INT 0 4
1 JMP 0 9
2 INT 1 4
3 LOD 1 -1
4 LIT 0 2
5 OPR 0 4
6 STO 1 3
7 LOD 1 3
8 RET 1 1
9 LIT 0 5
10 CAL 0 2
11 STO 0 3
12 LOD 0 3
13 OPR 0 14
14 RET 0 0
```

=== Program Output ===

10

Execution completed successfully.

Analisis: Menguji function dengan return value. Parameter n diakses dengan LOD 1 -1 (alamat negatif). Return value disimpan ke variabel lokal f (alamat 3) lalu di-load ke stack sebelum RET. Instruksi RET 1 1 memulihkan display level 1.

5.2.14 Kasus Uji 14: Boolean Literal

Berkas: pos_bool.txt

Masukan:

```
1 program BoolTest;  
2 var x: integer;  
3 begin  
4     if true then  
5         x := 1  
6     else  
7         x := 0;  
8     writeln(x);  
9 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4  
1 JMP 0 2  
2 LIT 0 1  
3 JPC 0 7  
4 LIT 0 1  
5 STO 0 3  
6 JMP 0 9  
7 LIT 0 0  
8 STO 0 3  
9 LOD 0 3  
10 OPR 0 14  
11 RET 0 0
```

=== Program Output ===

1

Execution completed successfully.

Analisis: Menguji boolean literal true yang di-inline sebagai LIT 0 1 (nilai konstanta true). Ini memvalidasi bahwa predefined identifiers (true, false, enum values) ditangani sebagai konstanta, bukan variabel yang perlu di-load dari memori.

6 Fitur Tambahan

Berikut fitur-fitur tambahan yang berhasil diimplementasikan di luar spesifikasi dasar:

1. **Deteksi Overflow Aritmatika:** Interpreter secara aktif memeriksa overflow pada operasi penjumlahan, pengurangan, dan perkalian bilangan bulat 32-bit. Jika terdeteksi overflow, eksekusi dihentikan dengan pesan `RuntimeError` yang jelas.
2. **Deteksi Division by Zero:** Sebelum melakukan operasi pembagian atau modulo, interpreter memeriksa apakah pembagi bernilai nol. Deteksi ini mencegah crash tidak terdefinisi pada level mesin.

3. **Bounds Checking pada Array:** Setiap akses elemen array (baik baca maupun tulis) disertai pemeriksaan batas indeks menggunakan instruksi `OPR 0 17` (`BOUNDS_CHECK`). Jika indeks di luar rentang yang dideklarasikan, eksekusi dihentikan dengan pesan error.
4. **Parameter Passing dengan Alamat Negatif:** Prosedur dan fungsi dengan parameter menggunakan skema alamat negatif (misalnya `LOD 1 -1`) untuk mengakses argumen yang diletakkan pada stack sebelum instruksi `CAL`. Skema ini memungkinkan pemanggilan prosedur dengan parameter tanpa memerlukan ruang variabel lokal tambahan.
5. **Return Value pada Function:** Function mengembalikan nilai melalui variabel lokal dengan nama yang sama dengan function itu sendiri. Return value di-load ke stack sebelum instruksi `RET` dieksekusi, memastikan nilai tersedia pada pemanggil setelah frame fungsi dibersihkan.

7 Penutup

7.1 Kesimpulan

Milestone 4 berhasil menyelesaikan rantai kompilasi bahasa Pascal sederhana (Arion) dengan mengimplementasikan *intermediate code generator* dan *stack machine interpreter*. Code generator mampu menerjemahkan *Decorated AST* dari tahap semantik menjadi instruksi P-Code (TAC) yang mencakup ekspresi, kontrol alur, subprogram, serta akses *array* dan *record*. Interpreter mengeksekusi instruksi-instruksi tersebut menggunakan manajemen *frame* (SL/DL/RA), *display* untuk resolusi *scope*, dan penanganan kesalahan runtime.

Beberapa aspek teknis menjadi kunci keberhasilan. Pertama, **parameter passing dengan alamat negatif** memungkinkan argumen yang didorong sebelum `CAL` diakses langsung oleh subprogram tanpa perlu memindahkan data. Kedua, **backpatching** memastikan instruksi lompatan (`JMP/JPC`) dapat dituju setelah seluruh blok kode dibangkitkan. Ketiga, **in-lining konstanta** (`true`, `false`, `enum`) memastikan ekspresi kondisional dapat dievaluasi tanpa alokasi memori runtime terpisah. Keempat, **bounds checking** pada array (`OPR 17`) mencegah korupsi memori dengan memvalidasi indeks sebelum akses.

Pengujian pada 14 kasus uji (12 positif dan 2 negatif) menunjukkan bahwa seluruh pipeline kompilasi—dari tokenisasi (M1), parsing (M2), analisis semantik (M3), hingga eksekusi kode perantara (M4)—bekerja secara terintegrasi tanpa regresi. Kode perantara yang dihasilkan konsisten dengan ekspektasi P-Code, dan kesalahan runtime ditangani dengan pesan informatif tanpa menyebabkan *crash*.

7.2 Saran

Berdasarkan keterbatasan yang ditemukan selama implementasi, beberapa saran perbaikan untuk pengembangan kompilator ini ke depan adalah sebagai berikut:

1. **Fase Optimasi:** Menambahkan fase optimasi kode perantara, seperti *constant folding* (misalnya `2 + 3` menjadi `LIT 5`), *dead code elimination*, dan *peephole optimization* untuk mengurangi jumlah instruksi yang tidak perlu.

2. **Dukungan Tipe Data Lengkap:** Melengkapi operasi runtime untuk tipe `real`, `char`, `string`, dan `enum` agar semua tipe yang telah didefinisikan pada tahap semantik (M3) dapat dieksekusi dengan benar.
3. **Pengujian Rekursi:** Melakukan pengujian eksplisit terhadap fungsi rekursif dan memperbaiki pengaturan *lexical level* pada `SemanticAnalyzer` (M3) agar rekursi dapat berjalan dengan benar.
4. **Penanganan Input yang Lebih Robust:** Menguji instruksi `read` dan `readln` dengan berbagai skenario input (termasuk input tidak valid) dan menambahkan mekanisme *buffering* yang lebih andal untuk pembacaan dari `stdin`.
5. **Ekspansi Arsitektur Target:** Mengembangkan *backend* tambahan yang mampu menghasilkan kode mesin untuk arsitektur tertentu (misalnya x86 atau RISC-V), sehingga kompilator tidak hanya terbatas pada mesin abstrak berbasis stack.

Appendix

Sumber Daya	Tautan
Repository GitHub	https://github.com/nicholaswisee/HBB-Tubes-IF2224-2026

Tabel 4: Tautan repository, deployment, dan diagram

Pembagian Tugas

Nama	NIM	Kontribusi
Nathanael Imandatua Manurung	13524021	25%
Nicholas Wise Saragih Sumbayak	13524037	25%
Kurt Mikhael Purba	13524065	25%
Rava Khoman Tuah Saragih	13524107	25%

Tabel 5: Pembagian kontribusi anggota kelompok